Spectral DNS of turbulence near the FFT roofline from Python

Walid Hemati
Independent researcher

Abstract

Pseudo-spectral direct numerical simulation (DNS) of incompressible turbulence spends almost all of its time in fast Fourier transforms. On a GPU those FFTs are memory-bound, so a time step has a hard lower bound: the number of transforms times the time of a standalone cuFFT call, plus the bytes the remaining pointwise work must move. We call this bound the *FFT roofline*. `pyspectral` is a Python package that solves the 2D vorticity equation and the 3D Navier–Stokes equations in rotational form on a periodic box. It uses 2/3-rule dealiasing, integrating-factor RK4 and CFL-adaptive time steps. Eleven interchangeable backend variants span an optimisation ladder: NumPy/SciPy, eager PyTorch, `torch.compile`, CuPy elementwise kernels, NVIDIA Warp, Triton and hand-written CUDA C compiled at run time. Every backend is validated against a NumPy oracle. The 2D Taylor–Green vortex is reproduced to 3.1×10^{-15} in double precision. RK4 converges at order 4.07, and the spatial error decays exponentially in the resolution. The 3D Taylor–Green vortex at $Re = 1600$ on a 256^3 grid reaches peak dissipation 0.01291 at $t = 8.90$; the 512^3 reference gives 0.01286 at $t = 8.97$. Three measures bring a step close to the FFT roofline: Basdevant's reformulation (4 instead of 5 FFTs per stage in 2D, 8 instead of 9 in 3D), one fused kernel on each side of the transforms, and in-place batched cuFFT plans. On an NVIDIA GeForce RTX 3070 Laptop GPU the fastest variant (`cupy-cuda-c/fused-inplace`) advances a 8192^2 grid in 122.63 ms per RK4 step, 99 % of the FFT roofline, and a 384^3 grid in 182.44 ms (103 %).

1 Introduction

Fourier pseudo-spectral methods have been the reference discretisation for DNS of homogeneous turbulence since the work of Orszag and Patterson [1, 2] and Rogallo [3]. They are still the method of choice for the largest simulations [4–7]. Each time step is a sequence of forward and inverse multi-dimensional FFTs, joined by cheap pointwise operations in physical and spectral space. On modern GPUs large FFTs are limited by memory bandwidth, not arithmetic. A pseudo-spectral code therefore cannot run faster than the FFT library allows. Its efficiency is set by two things: how many FFTs it needs per step, and how little extra memory traffic surrounds them.

This paper asks whether that hardware limit can be reached from Python. Mortensen and Langtangen [8] showed that a pure-Python/NumPy pseudo-spectral solver on CPUs can nearly match a C++ implementation. Dedalus [9] is a general Python spectral framework. Neither study measures a single GPU against its own roofline. We do that, following the roofline methodology of Williams et al. [10]. The ceiling is not peak FLOP/s or raw bandwidth. It is the measured time of the standalone cuFFT calls the algorithm cannot avoid. Our contributions are:

- A pseudo-spectral Navier–Stokes solver, `pyspectral`: 2D vorticity form and 3D rotational form with Leray projection, 2/3-rule (or phase-shift) dealiasing, Lawson integrating-factor RK4 [11], CFL-adaptive steps, and constant-power forcing. Eleven backend variants share one interface and one test suite.

- An FFT-count-minimal formulation following Basdevant [12]: 4 real-to-complex transforms per RK stage in 2D and 8 in 3D, with a fused schedule. Each stage runs exactly two pointwise kernels, the transforms are batched and in place, and the integrating factor comes from a separable table.
- An *FFT roofline* for pseudo-spectral steps. It combines the measured standalone cuFFT time for the exact batched transforms of a stage with a minimal pointwise-traffic model. We report every backend as a fraction of it.
- Validation against exact solutions and against the 512^3 reference DNS of the Taylor–Green vortex at $Re = 1600$ [13, 14], plus forced 2D and 3D spectra.
- Same-machine head-to-heads with JAX-CFD, spectralDNS and Dedalus (section 6.1).

2 Background and related work

Pseudo-spectral DNS. In the Fourier–Galerkin/collocation method the nonlinear term is evaluated in physical space, so quadratic products alias. Orszag's 2/3 rule removes aliasing exactly for quadratic terms by truncating $|k_i| \geq N/3$ [1]. Phase shifts with spherical truncation [2, 3] keep more modes at the price of extra transforms or randomisation. Textbook treatments are given by Canuto et al. [15, 16]. Basdevant [12] introduced formulations of the nonlinear term that need fewer transforms. Exponential and integrating-factor integrators treat the stiff linear term exactly [11, 17]. Record DNS runs pushed Fourier methods to 4096^3 on the Earth Simulator [4, 5]. More recent work runs them on thousands of GPUs [6, 7]. Those codes target aggregate throughput on supercomputers. Our claim is narrower: per-GPU hardware efficiency from Python.

Python for spectral DNS. Mortensen and Langtangen [8] report that their own C++ port is about 30 % faster than their pure-Python solver in serial runs on a laptop CPU. On a Blue Gene/P their Cython-optimised version performed as well as C++ across the processor counts they tested. Dedalus [9] generalises spectral methods to many equations and bases. Neither targets GPUs. On the GPU side, Python now offers several kernel-generation routes: PyTorch [18] and its compiler [19], Triton [20], CuPy [21] and NVIDIA Warp [22]. All of them can call cuFFT [23]. CPU FFTs use pocketfft through SciPy [24, 25], in the role FFTW plays in compiled codes [26].

3 Method

3.1 Equations

We solve the incompressible Navier–Stokes equations on $[0, 2\pi)^d$ with integer wavenumbers $\mathbf{k}$. In 2D we evolve the vorticity $\omega = \partial_x v - \partial_y u$,

$$\partial_t \omega + \nabla \cdot (\mathbf{u}\,\omega) = \nu \nabla^2 \omega - \alpha\,\omega + f, \qquad \hat{\psi} = \hat{\omega}/k^2, \quad \hat{u} = \mathrm{i}k_y \hat{\psi}, \quad \hat{v} = -\mathrm{i}k_x \hat{\psi}. \tag{1}$$

In 3D we evolve the velocity in rotational form,

$$\partial_t \hat{\mathbf{u}} = \mathsf{P}(\mathbf{k})\, \widehat{\mathbf{u} \times \boldsymbol{\omega}} - \nu k^2 \hat{\mathbf{u}} + \hat{\mathbf{f}}, \qquad \mathsf{P}_{ij} = \delta_{ij} - k_i k_j / k^2, \tag{2}$$

where the Leray projector P absorbs the pressure and the gradient of the kinetic energy.

3.2 Fewer transforms per stage

The textbook 2D advective form $\mathbf{u}\cdot\nabla\omega$ needs four inverse transforms $(u, v, \omega_x, \omega_y)$ and one forward transform. The 3D rotational form needs six inverse $(\mathbf{u}, \boldsymbol{\omega})$ and three forward. Following Basdevant [12], for divergence-free $\mathbf{u}$ we use the identity (checked by direct expansion and numerically in our tests)

$$\nabla\cdot(\mathbf{u}\,\omega) = \partial_x\partial_y\,(v^2-u^2) + (\partial_x^2-\partial_y^2)\,(uv), \tag{3}$$

which needs only the inverse transforms of u, v and the forward transforms of v^2-u^2 and uv: 4 FFTs. In 3D we write the nonlinear term in divergence form with the traceless stress $T_{ij} = u_iu_j - \frac{1}{3}|\mathbf{u}|^2\delta_{ij}$. The trace is a gradient and the projection removes it. Since $T_{33} = -T_{11}-T_{22}$, the term $\hat{N}_i = -\mathrm{i}\,\mathsf{P}_{ij}k_l\hat{T}_{jl}$ needs 3 inverse and 5 forward transforms (8 instead of 9). With the 2/3 rule all products are computed exactly on the retained modes, so the rotational and the divergence form give identical results up to round-off. Our tests confirm this at the 10^{-16} level.

3.3 Dealiasing, time integration and forcing

We keep modes with $3|k_i| < N$ in every direction (cubic 2/3 truncation). The NumPy and eager PyTorch backends also offer random phase shifts with spherical truncation $|\mathbf{k}| < \sqrt{2}N/3$. The shift is redrawn each step and moved by half a grid cell between the first and the second pair of RK stages, after Rogallo [3]. The linear operator

$$L(\mathbf{k}) = -\nu k^2 - \alpha\,[k \le k_\alpha] + A\,[k_{\mathrm{lo}} \le k \le k_{\mathrm{hi}}] \tag{4}$$

combines viscosity, linear drag (uniform if $k_\alpha = \infty$, otherwise acting on large scales only) and band-limited linear forcing. The forcing is a spectral variant of the linear forcing of Lundgren [27, 28]. A constant-power controller resets $A = P/(2E_{\mathrm{band}})$ every few steps. With $E = e^{L\Delta t}$ and $E_{1/2} = e^{L\Delta t/2}$, the Lawson integrating-factor RK4 [11] reads

$$\begin{aligned} k_1 &= N(u_n), & k_2 &= N(E_{1/2}(u_n + \tfrac{\Delta t}{2}k_1)),\\ k_3 &= N(E_{1/2}u_n + \tfrac{\Delta t}{2}k_2), & k_4 &= N(Eu_n + \Delta t\,E_{1/2}k_3),\\ u_{n+1} &= Eu_n + \tfrac{\Delta t}{6}(Ek_1 + 2E_{1/2}(k_2+k_3) + k_4). \end{aligned} \tag{5}$$

It integrates the linear term exactly and needs one accumulator besides the state. The step size follows $\Delta t = C\,\Delta x/\sum_i \max|u_i|$ with $C = 0.5$, re-evaluated every 10 steps.

4 Implementation: the optimisation ladder

All backends implement the same `SpectralSolver` interface and are listed in a registry (table 2). The NumPy/SciPy backend is a direct transcription of the equations. It is the correctness oracle and runs in CI. The GPU backends form a ladder:

torch-eager ports the NumPy code to `torch.fft`. The variants use complex-to-complex transforms on the full spectrum (`c2c-textbook`), real-to-complex transforms (`r2c-textbook`), and the minimal FFT count (`r2c-minfft`). Every arithmetic operation launches its own kernel and allocates a temporary.

torch-compile writes the two pointwise phases as plain PyTorch on real views of the complex buffers. TorchInductor compiles them into fused Triton kernels. Stage coefficients and Δt come from a device tensor, so adaptive steps never trigger recompilation.

cupy fuses each phase into one `ElementwiseKernel`. FFTs go through the SciPy-compatible API of CuPy, with cached plans and out-of-place transforms.

warp, triton use hand-written fused kernels in NVIDIA Warp and Triton. They run on pre-allocated torch buffers with `torch.fft` transforms written into fixed output buffers. The forward-FFT output doubles as the next inverse-FFT input.

cupy-cuda-c is the hand-written reference: CUDA C compiled at run time by NVRTC through CuPy's `RawModule`, with batched *in-place* cuFFT plans (`cufftPlanMany` with padded real layout) created through CuPy's low-level plan interface. The `lean` variant loops over single-field plans to save memory. The `graph` variant captures one time step in a CUDA graph.

Fused schedule. One RK stage of the fused backends is *C2R (batched, in place)* → *physical kernel* → *R2C (batched, in place)* → *spectral kernel*. The physical kernel reads u, v (3D: $\mathbf{u}$) and writes the products in place over them. The spectral kernel reads the transformed products, assembles $\hat{N}$, applies the dealiasing mask and the projection, and performs the IF-RK4 stage update on the state and the accumulator. It then writes the next stage's FFT inputs: the streamfunction velocities in 2D, $\hat{\mathbf{u}}$ (and $\hat{\boldsymbol{\omega}}$ for the rotational form) in 3D, already scaled by N^{-d} for the unnormalised cuFFT inverse. No separate "pre" kernel exists. Viscous factors cost no transcendental functions because $e^{-\nu k^2 \theta \Delta t} = t_\theta[|k_x|]\, t_\theta[|k_y|]\, t_\theta[|k_z|]$ separates into a 1D table, with drag and band forcing applied as two scalar factors. This matters in fp64, where `exp` is expensive on consumer GPUs. The memory footprint is the state, the accumulator and $\max(n_{\text{inv}}, n_{\text{fwd}})$ work fields: 4 spectral fields in 2D, 11 in 3D.

FFT roofline and traffic model. Let t_{FFT} be the measured time of the batched C2R plus R2C transforms of one stage. We use the faster of in-place CuPy plans and out-of-place `torch.fft`, timed standalone on identical shapes. Let B be the minimal pointwise traffic per step. The physical kernel reads the n_{inv} inverse outputs and writes the n_{fwd} products. The spectral kernel reads the n_{fwd} transformed products and the c state components, writes n_{inv} next inputs, and reads and writes the accumulator ($7c$ field transfers per step). The FFT-roofline step time is

$$T_{\text{roof}} = 4\, t_{\text{FFT}} + B/B_{\text{copy}}, \qquad B_{\text{copy}} = 361\,\text{GB/s}, \tag{6}$$

where B_{copy} is the device-to-device copy bandwidth measured for the whole suite (the datasheet value is 448 GB/s). We report $T_{\text{roof}}/T_{\text{step}}$ as "% of FFT roofline". The model is additive and ignores overlap and cache reuse between consecutive kernels. A fused solver can therefore come in slightly above 100 %, for example when part of a field is still in the 4 MB L2 cache, or when its pointwise kernels stream faster than the copy bandwidth used in the model. For the conventional bandwidth view we also count each FFT as one read and one write of its data, which is a lower bound because real 2D/3D FFTs take several passes. Adding B gives an effective bandwidth that can only be under-, never over-estimated. Throughput is given in Mpts/s, grid points advanced by one full RK4 step per second.

5 Validation

Exact solutions and convergence. The 2D Taylor–Green vortex $\omega = 2 \sin x \sin y$ is a steady Euler solution, so it decays as $e^{-2\nu t}$. Integrating factors make IF-RK4 exact for it. All 11 variants reproduce it to 3.1×10^{-15} (fp64) and 7.0×10^{-6} (fp32). With the viscous term treated explicitly,

classical RK4 converges at order 4.07 (fig. 1). For a nonlinear, non-band-limited flow, IF-RK4 self-convergence gives order 3.99. The spatial error against an $N = 192$ solution decays exponentially, to 6.1×10^{-11} at $N = 64$. Every GPU variant matches the fp64 NumPy oracle across 64 cases (2D/3D, both 3D forms, with drag, forcing, a changed forcing amplitude and a changed Δt) to 1.3×10^{-15} in fp64 and 8.3×10^{-7} in fp32. Further tests check inviscid energy conservation, the 2D energy budget $\mathrm{d}E/\mathrm{d}t = -2\nu Z$, a divergence-free velocity, and agreement of phase-shift with 2/3-rule dealiasing.

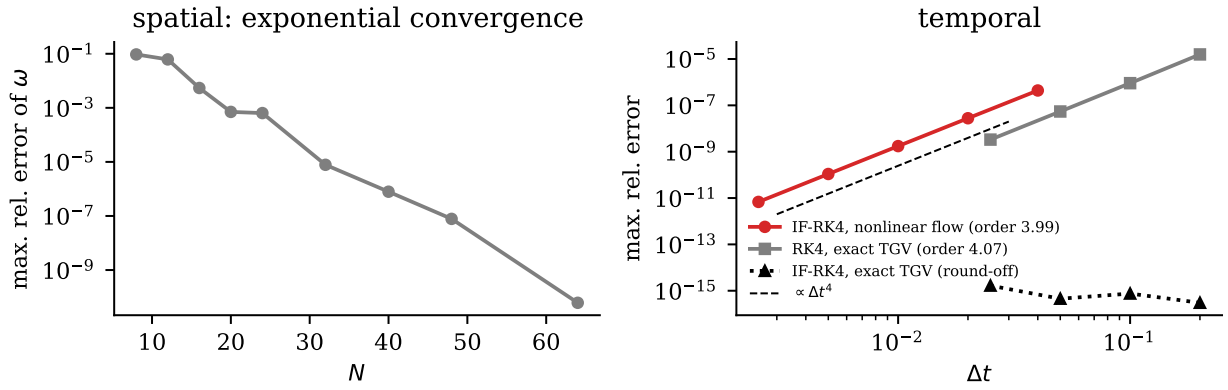


Figure 1: Left: spectral convergence in N for a smooth 2D flow (error against $N = 192$). Right: temporal convergence. Classical RK4 on the exact TGV decay, IF-RK4 self-convergence on a nonlinear flow, and IF-RK4 on the TGV, which is exact to round-off.

3D Taylor–Green vortex at $Re = 1600$. The initial condition is $u = \sin x \cos y \cos z$, $v = -\cos x \sin y \cos z$, $w = 0$ with $\nu = 1/1600$, integrated to $t = 20$. This is the benchmark of the high-order CFD workshops [13, 29], originally studied by Brachet et al. [30]. We compare the dissipation rate $\varepsilon = 2\nu Z$ with the 512^3 pseudo-spectral reference distributed for workshop case C3.5. Its documentation attributes the data to a dealiased pseudo-spectral code from Université catholique de Louvain and cites van Rees et al. [14] for an independent spectral convergence study. The comparison is in table 1 and fig. 2. At 256^3 the peak is 0.01291 at $t = 8.90$, against 0.01286 at $t = 8.97$ in the reference (0.43 % difference). The largest deviation over $t \in [0, 20]$ is 5.94 % of the peak value. It occurs after the peak, during the breakdown into small-scale turbulence. Coarser grids over-predict the peak, as expected for under-resolved Galerkin truncations, where energy piles up near the cutoff. The peak error falls monotonically with N (table 1). The post-peak deviation does not: it is smaller at 192^3 than at 256^3. Possible causes are the remaining resolution gap to the 512^3 reference, single precision (which can break the flow's discrete symmetries earlier than a double-precision run), and the different time integrator of the reference (low-storage RK3). We have not separated these effects.

Forced turbulence. Constant-power band forcing gives statistically stationary turbulence (fig. 3). In 2D ($N = 512$, forcing at $k \approx 4$, drag on $k \leq 2.5$) the enstrophy-cascade range has $E(k) \sim k^{-3.83}$ for $15 \leq k \leq 50$. That is steeper than Kraichnan's k^{-3} [31, 32], as is common at this modest scale separation with large-scale friction. In 3D ($N = 192$, $k_{\max}\eta = 1.48$) the Taylor-scale Reynolds number is only $Re_\lambda = 80$. The compensated spectrum therefore shows only a short plateau just above the forcing scale, followed by the dissipation range, and no extended

Table 1: 3D Taylor–Green vortex at $Re = 1600$ (fp32, `cupy-cuda-c`, CFL 0.5) against the 512^3 reference (peak 0.01286 at $t = 8.97$).

grid	peak ε	t_{peak}	peak error [%]	max $\lvert\Delta\varepsilon\rvert/\varepsilon_{\max}$ [%]	steps
64^3	0.01340	9.20	+4.18	14.91	1167
128^3	0.01376	8.90	+7.01	9.48	2109
192^3	0.01298	8.85	+0.98	3.09	3048
256^3	0.01291	8.90	+0.43	5.94	4075

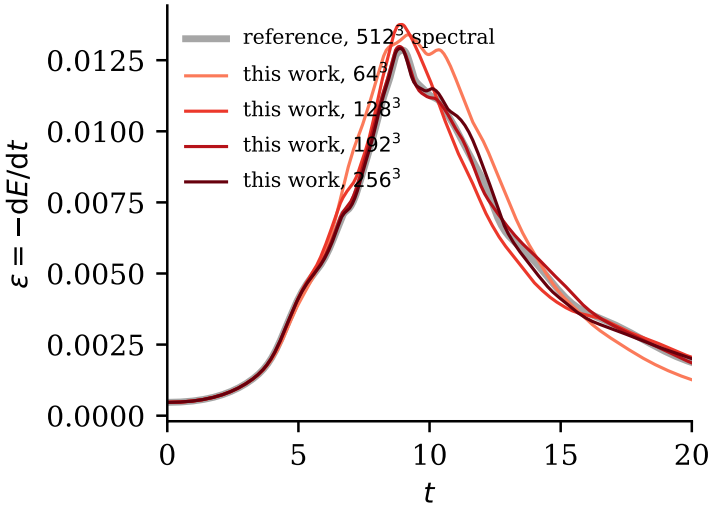


Figure 2: Kinetic-energy dissipation rate of the Taylor–Green vortex at $Re = 1600$ against the 512^3 spectral reference.

$k^{-5/3}$ range [33]. The local slope over $3 \le k \le 8$ is -1.48. Both runs serve as end-to-end tests of the forcing, drag and diagnostics machinery. They are not converged turbulence statistics.

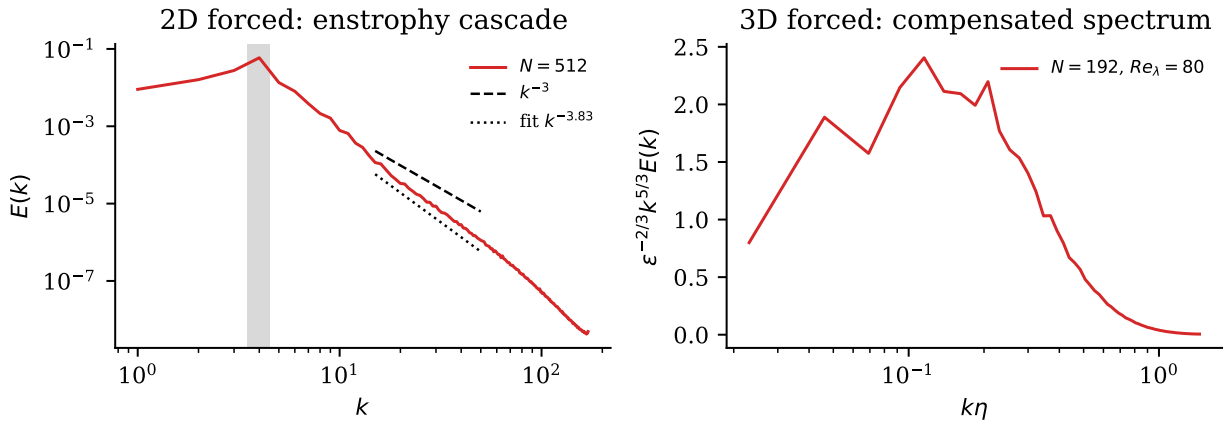


Figure 3: Time-averaged energy spectra of forced turbulence. Left: 2D (grey band: forcing). Right: 3D, compensated by $\varepsilon^{-2/3}k^{5/3}$.

6 Performance

All performance numbers come from `bench/run.py` on an NVIDIA GeForce RTX 3070 Laptop GPU (GA104, 40 SMs, 8 GB GDDR6, datasheet bandwidth 448 GB/s, measured copy bandwidth 361 GB/s). We report the median of 5 repeats, each lasting at least 0.5 s, after a warm-up that covers JIT compilation, plan creation and autotuning.

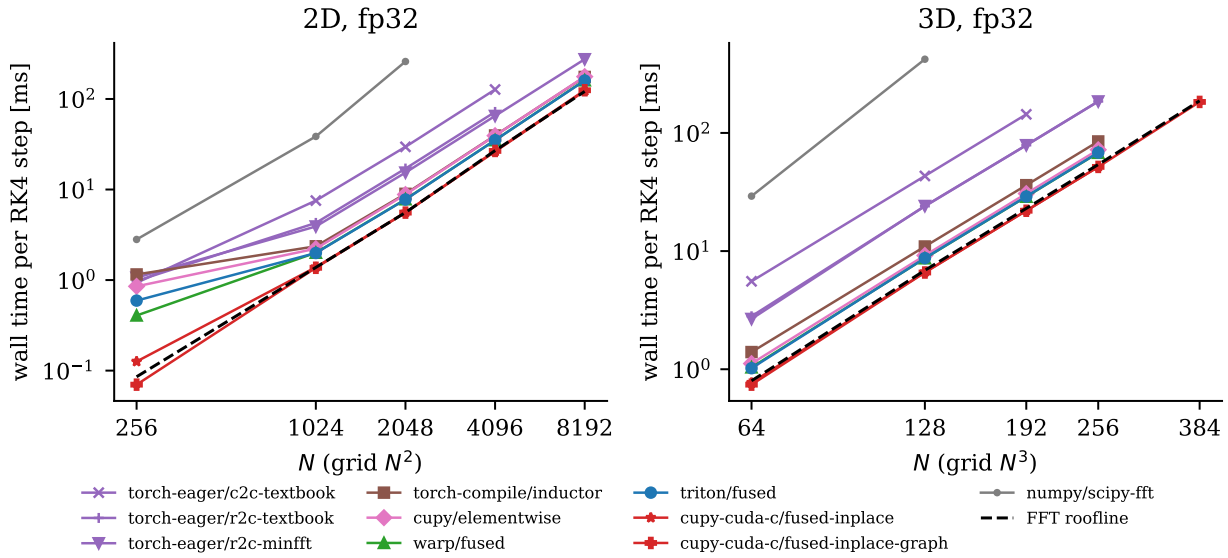


Figure 4: Wall time per RK4 step against grid size (fp32), with the FFT roofline (6) (dashed).

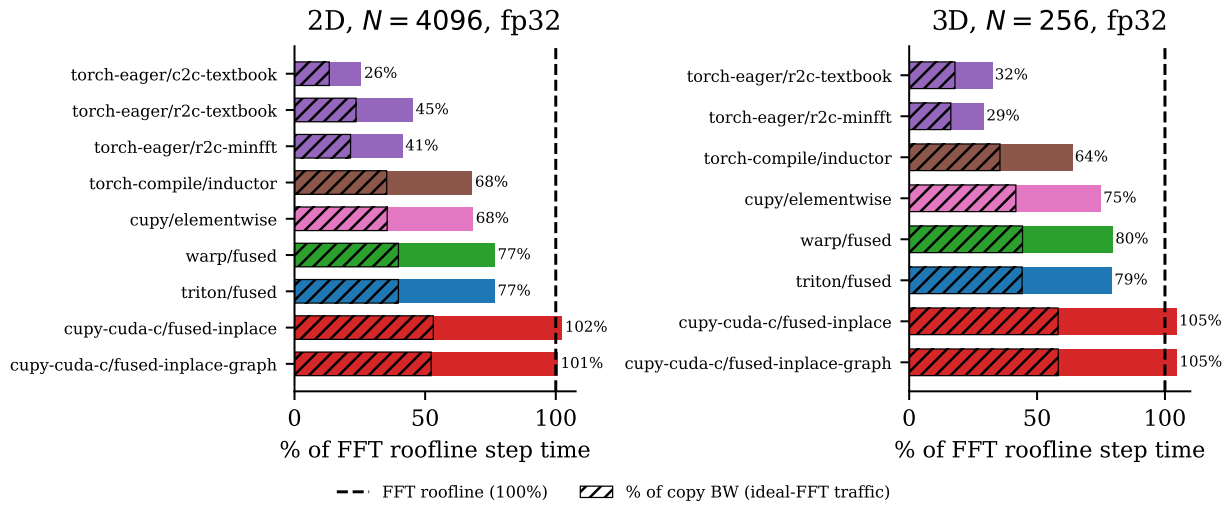


Figure 5: Fraction of the FFT roofline reached by each variant at the largest grid (solid). The hatched bars show effective bandwidth as a percentage of 361 GB/s under the ideal single-pass FFT traffic model. Dashed line: the roofline.

Table 2: Step times at the largest grids (fp32). "FFT" is the number of real FFTs per RK4 step; "GB/s" and "% copy" use the ideal traffic model; "% roof" is $T_{\text{roof}}/T_{\text{step}}$.

grid	variant	FFT	ms/step	Mpts/s	GB/s	% copy	% roof
2D 4096^2	`torch-eager/c2c-textbook`	20	127.20	132	48	13	26
2D 4096^2	`torch-eager/r2c-textbook`	20	71.90	233	85	24	45
2D 4096^2	`torch-eager/r2c-minfft`	16	65.01	258	77	21	41
2D 4096^2	`torch-compile/inductor`	16	39.52	424	127	35	68
2D 4096^2	`cupy/elementwise`	16	39.34	426	128	35	68
2D 4096^2	`triton/fused`	16	35.07	478	144	40	77
2D 4096^2	`warp/fused`	16	35.06	479	144	40	77
2D 4096^2	`cupy-cuda-c/fused-inplace-graph`	16	26.68	629	189	52	101
2D 4096^2	`cupy-cuda-c/fused-inplace`	16	26.25	639	192	53	102
3D 256^3	`torch-eager/r2c-minfft`	32	184.83	91	59	16	29
3D 256^3	`torch-eager/r2c-textbook`	36	184.68	91	65	18	32
3D 256^3	`torch-compile/inductor`	32	84.71	198	128	35	64
3D 256^3	`cupy/elementwise`	32	72.09	233	151	42	75
3D 256^3	`triton/fused`	32	68.15	246	159	44	79
3D 256^3	`warp/fused`	32	67.85	247	160	44	80
3D 256^3	`cupy-cuda-c/fused-inplace`	32	51.67	325	210	58	105
3D 256^3	`cupy-cuda-c/fused-inplace-graph`	32	51.59	325	210	58	105

The ladder. Figures 4 and 5 and table 2 show the progression. Eager PyTorch pays for every intermediate array. At 4096^2, the largest 2D grid on which every rung ran, the fastest variant (`cupy-cuda-c/fused-inplace`) is 2.5× faster than eager PyTorch with the same FFT count, and 4.8× faster than the complex-to-complex textbook version. Against the multithreaded NumPy/SciPy reference on 16 CPU threads, the fastest GPU variant is 47× faster at $N = 2048$ in 2D and 64× faster at $N = 128$ in 3D. Fusing the pointwise phases into one kernel each (Inductor, CuPy, Warp, Triton) removes most of the gap. What remains is mostly the transforms. For identical shapes, the out-of-place `torch.fft` and `cupyx.scipy.fft` calls are measurably slower than in-place batched cuFFT plans; both timings are stored with the results. At the largest grids the fastest variants reach 99 % of the FFT roofline in 2D (`cupy-cuda-c/fused-inplace`, 8192^2) and 103 % in 3D (`cupy-cuda-c/fused-inplace-lean`, 384^3), where the FFTs alone take 69 % of the step. At 256^3 the rotational form (9 FFTs) takes 1.13× the step time of the Basdevant form (8 FFTs) in the CUDA C backend. CUDA graphs pay off only on small grids, where launch latency dominates.

Precision. On this consumer GPU, double-precision arithmetic runs at a small fraction of single-precision throughput. Double-precision FFTs therefore become compute-bound: at 8192^2 an fp64 step of the CUDA C backend takes 5.6× as long as an fp32 step, much more than the 2× that the doubled byte count alone would cause. Production DNS on such hardware is therefore done in fp32. The Taylor–Green results above use fp32.

6.1 Head-to-head with established codes

Choice of counterparts. We compare with three established open-source codes, each in its own environment and driven by `bench/counterparts.py`. (a) The spectral module of JAX-CFD [34, 35] (`jax_cfd.spectral`, JAX [36] 0.11.2, commit `469d789`) solves the 2D vorticity equation pseudo-spectrally on the GPU. We run its stock time stepper, a Crank–Nicolson/low-storage RK4 with five stages [37] (`cn-rk4`, 25 FFTs per step), and its right-hand side inside our Lawson IF-RK4 (`if-rk4`,

20 FFTs: the same algorithm as our advective form). (b) spectralDNS [8] 1.4.0 (shenfun 4.3.0, mpi4py-fft 2.0.6 [38, 39], MPICH 5.0.2) is the Python DNS code whose CPU speed Mortensen and Langtangen compared with C++. It runs on the AMD Ryzen 7 5800H of the same laptop in the rotational form with classical RK4 and the 2/3 rule (36 FFTs per step, as our rotational form), slab-decomposed over MPI ranks, with its `numba` kernels (the Cython ones do not build against NumPy 2). (c) Dedalus [9] (`gpu` branch, 3.0.5, commit `2389e5d`, CuPy 14.2.0) is, to our knowledge, the only established Python spectral framework with GPU support for 3D periodic problems. We keep $M = 2N/3$ Fourier modes per axis with 3/2 padding, which retains exactly the modes and product grid of the 2/3 rule on N^3, solve $\partial_t \mathbf{u} + \nabla P - \nu \nabla^2 \mathbf{u} = -\mathbf{u} \cdot \nabla \mathbf{u}$, $\nabla \cdot \mathbf{u} = 0$ (its cross product is not yet ported to the GPU) with the 4-stage IMEX scheme RK443 (Dedalus has no classical RK4) at constant Δt.

Agreement. On the exact 2D Taylor–Green decay both JAX-CFD steppers stay within 4.6×10^{-12} (relative L_2, fp64). For decaying 2D turbulence at $N = 256$ the state after 200 steps differs from our NumPy oracle by 2.3×10^{-6} with IF-RK4 and 2.3×10^{-6} with JAX-CFD's own stepper; JAX-CFD's brick-wall filter is shifted by one row of modes at the k_x edge, so an under-resolved flow would show a larger gap. For the 3D Taylor–Green vortex at $Re = 1600$, the energy and enstrophy of spectralDNS after 50 steps on 128^3 match our explicit-RK4 rotational run to 4.4×10^{-16}. Dedalus matches our IF-RK4 run on 96^3 to 2.0×10^{-9} after 50 steps, consistent with its third-order time error.

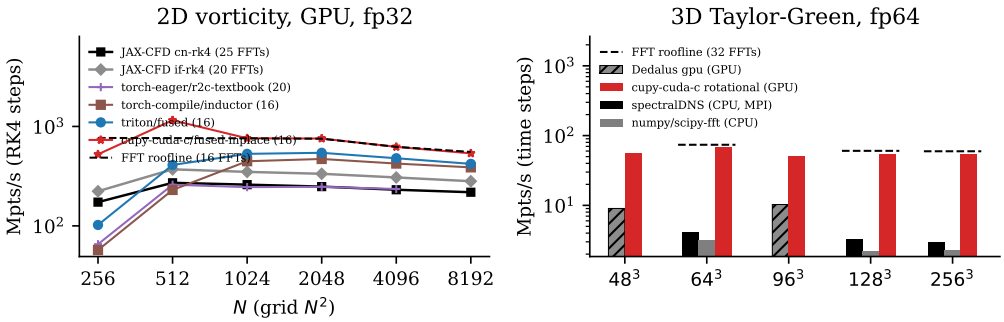


Figure 6: Head-to-heads on one machine. Left: 2D, JAX-CFD (black, grey) and pyspectral variants (FFTs per step in brackets). Right: 3D Taylor–Green, fp64: spectralDNS (CPU, best rank count), NumPy oracle (CPU), Dedalus (GPU), CUDA C backend (rotational). Dashed: FFT roofline (Basdevant).

Speed. Figure 6 shows the results; every run is listed with its notes in `results/counterparts.json`. At 8192^2 (fp32) `cupy-cuda-c/fused-inplace` needs 125.15 ms per step, 2.5× less than JAX-CFD with its own stepper (307.87 ms) and 1.9× less than JAX-CFD's right-hand side in the same IF-RK4 (238.49 ms). Per transform, our step processes 1.5× as many FFT points per second (8.6 vs. 5.6 Gpts/s), so the gap is not only the FFT count. XLA itself fuses well: JAX-CFD's IF-RK4 is n/a× faster than eager PyTorch running the identical algorithm (n/a ms). In 3D at 256^3 (fp64) spectralDNS on 8 MPI ranks takes 5737.5 ms per step and our NumPy oracle with `scipy.fft` on 16 threads 7293.5 ms; the CUDA C backend in the same rotational form is 18× faster than spectralDNS. At 96^3 (64^3 modes, fp64) Dedalus needs 86.05 ms per step and 2431 MB of device memory; the CUDA C backend needs 17.37 ms (5.0× less; 5.6× in Basdevant form). Dedalus

solves general equations through sparse linear solves and transforms the full gradient tensor; the ratio is the price of generality. All ratios are per step at equal Δt; the schemes differ in stages, order and stability limit, and spectralDNS's CPU timings depend on rank placement and on the `numba` rather than Cython kernels.

7 Discussion

At large N the pointwise work is almost free once fused, so the FFT library is the ceiling. The two remaining levers are fewer transforms and transforms that move less data. Basdevant's formulation takes care of the first. For the second, in-place batched plans are faster than the out-of-place calls of the high-level APIs and need about half the work memory. Below the roofline, further gains need transform-level fusion. Examples are cuFFT callbacks that fold the pointwise kernels into the FFT load and store stages, or custom FFTs that keep pencils in shared memory, as large-scale codes do with their own transposes [6]. Neither is available from pure Python without a host compiler.

Threats to validity. (i) All results come from one laptop GPU. Laptop parts throttle under sustained load, and clocks were not pinned, so step times can drift by several percent between runs. (ii) The roofline uses the suite-wide copy bandwidth of 361 GB/s. Our own copy measurement in the same run is stored in the results file. (iii) The CUDA C backend is compiled at run time by NVRTC rather than offline by nvcc. Default optimisation flags and link-time options can differ. (iv) Our FFT ceiling is cuFFT as shipped with the installed CUDA libraries. A faster FFT would move the roofline. (v) Development timings were taken on a shared GPU; every number in this paper comes from the `--full` run on an idle GPU. (vi) At 384^3 only the lean in-place CUDA C variant fits into the 8 GB of the GPU; the other variants run out of memory and are listed as skipped in the results file.

8 Conclusion

A Python package can run Fourier pseudo-spectral DNS close to the speed of the FFTs it cannot avoid. Minimising the FFT count, fusing all pointwise work into two kernels per stage, and using in-place batched cuFFT plans bring a step to 99 % (2D) and 103 % (3D) of the FFT roofline on a single consumer GPU. Exact solutions and the $Re = 1600$ Taylor–Green reference validate the physics. High-level routes (Inductor, CuPy, Warp, Triton) reach 69–76 % of the roofline at 8192^2 and 64–80 % at 256^3; hand-written CUDA C closes the remaining gap.

Code availability

Code, benchmark data and scripts to rebuild every figure and number in this paper: `https://github.com/walidhemati/pyspectral` (MIT licence).

References

[1] Steven A. Orszag. On the elimination of aliasing in finite-difference schemes by filtering high-wavenumber components. *Journal of the Atmospheric Sciences*, 28(6):1074, 1971. doi: 10.1175/1520-0469(1971)028<1074:OTEOAI>2.0.CO;2.

[2] G. S. Patterson and Steven A. Orszag. Spectral calculations of isotropic turbulence: Efficient removal of aliasing interactions. *Physics of Fluids*, 14(11):2538–2541, 1971. doi: 10.1063/1.1693365.

[3] Robert S. Rogallo. Numerical experiments in homogeneous turbulence. NASA Technical Memorandum 81315, NASA Ames Research Center, September 1981. URL https://ntrs.nasa.gov/citations/19810022965. NTRS document ID 19810022965.

[4] M. Yokokawa, K. Itakura, A. Uno, T. Ishihara, and Y. Kaneda. 16.4-Tflops direct numerical simulation of turbulence by a Fourier spectral method on the Earth Simulator. In *Proceedings of the ACM/IEEE SC 2002 Conference (SC'02)*, page 50. IEEE, 2002. doi: 10.1109/SC.2002.10052.

[5] Takashi Ishihara, Toshiyuki Gotoh, and Yukio Kaneda. Study of high-Reynolds number isotropic turbulence by direct numerical simulation. *Annual Review of Fluid Mechanics*, 41:165–180, 2009. doi: 10.1146/annurev.fluid.010908.165203.

[6] Kiran Ravikumar, David Appelhans, and P. K. Yeung. GPU acceleration of extreme scale pseudo-spectral simulations of turbulence using asynchronism. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*, pages 1–22, New York, NY, USA, 2019. ACM. doi: 10.1145/3295500.3356209.

[7] P. K. Yeung, Kiran Ravikumar, Stephen Nichols, and Rohini Uma-Vaideswaran. GPU-enabled extreme-scale turbulence simulations: Fourier pseudo-spectral algorithms at the exascale using OpenMP offloading. *Computer Physics Communications*, 306:109364, 2025. doi: 10.1016/j.cpc.2024.109364.

[8] Mikael Mortensen and Hans Petter Langtangen. High performance Python for direct numerical simulations of turbulent flows. *Computer Physics Communications*, 203:53–65, 2016. doi: 10.1016/j.cpc.2016.02.005.

[9] Keaton J. Burns, Geoffrey M. Vasil, Jeffrey S. Oishi, Daniel Lecoanet, and Benjamin P. Brown. Dedalus: A flexible framework for numerical simulations with spectral methods. *Physical Review Research*, 2(2):023068, 2020. doi: 10.1103/PhysRevResearch.2.023068.

[10] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[11] J. Douglas Lawson. Generalized Runge–Kutta processes for stable systems with large Lipschitz constants. *SIAM Journal on Numerical Analysis*, 4(3):372–380, 1967. doi: 10.1137/0704033.

[12] C. Basdevant. Technical improvements for direct numerical simulation of homogeneous three-dimensional turbulence. *Journal of Computational Physics*, 50(2):209–214, 1983. doi: 10.1016/0021-9991(83)90064-5.

[13] Z. J. Wang, Krzysztof Fidkowski, Rémi Abgrall, Francesco Bassi, Doru Caraeni, Andrew Cary, Herman Deconinck, Ralf Hartmann, Koen Hillewaert, H. T. Huynh, Norbert Kroll, Georg May, Per-Olof Persson, Bram van Leer, and Miguel Visbal. High-order CFD methods: Current status and perspective. *International Journal for Numerical Methods in Fluids*, 72(8):811–845, 2013. doi: 10.1002/fld.3767.

[14] Wim M. van Rees, Anthony Leonard, D. I. Pullin, and Petros Koumoutsakos. A comparison of vortex and pseudo-spectral methods for the simulation of periodic vortical flows at high Reynolds numbers. *Journal of Computational Physics*, 230(8):2794–2805, 2011. doi: 10.1016/j.jcp.2010.11.031.

[15] Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods in Fluid Dynamics*. Springer, Berlin, Heidelberg, 1988. doi: 10.1007/978-3-642-84108-8.

[16] Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods: Fundamentals in Single Domains*. Scientific Computation. Springer, Berlin, Heidelberg, 2006. ISBN 978-3-540-30725-9. doi: 10.1007/978-3-540-30726-6.

[17] S. M. Cox and P. C. Matthews. Exponential time differencing for stiff systems. *Journal of Computational Physics*, 176(2):430–455, 2002. doi: 10.1006/jcph.2002.6995.

[18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[19] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th*

ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), pages 929–947, New York, NY, USA, 2024. ACM. doi: 10.1145/3620665.3640366.

[20] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, pages 10–19, New York, NY, USA, 2019. ACM. doi: 10.1145/3315508.3329973.

[21] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[22] Miles Macklin. Warp: A high-performance Python framework for GPU simulation and graphics. https://github.com/NVIDIA/warp, March 2022. NVIDIA GPU Technology Conference (GTC).

[23] NVIDIA Corporation. cuFFT library documentation. https://docs.nvidia.com/cuda/cufft/. Version 13.4 documentation at time of access.

[24] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[25] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[26] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93 (2):216–231, 2005. doi: 10.1109/JPROC.2004.840301.

[27] T. S. Lundgren. Linearly forced isotropic turbulence. Annual research briefs, Center for Turbulence Research, Stanford University, 2003. URL https://web.stanford.edu/group/ctr/ResBriefs03/lundgren.pdf.

[28] Carlos Rosales and Charles Meneveau. Linear forcing in numerical simulations of isotropic turbulence: Physical space implementations and convergence properties. *Physics of Fluids*, 17(9):095106, 2005. doi: 10.1063/1.2047568.

[29] James R. DeBonis. Solutions of the Taylor–Green vortex problem using high-resolution explicit finite difference methods. In *51st AIAA Aerospace Sciences Meeting including the New Horizons Forum and Aerospace Exposition*, Grapevine (Dallas/Ft. Worth Region), Texas, 2013. American Institute of Aeronautics and Astronautics. doi: 10.2514/6.2013-382. AIAA Paper 2013-0382.

[30] Marc E. Brachet, Daniel I. Meiron, Steven A. Orszag, B. G. Nickel, Rudolf H. Morf, and Uriel Frisch. Small-scale structure of the Taylor–Green vortex. *Journal of Fluid Mechanics*, 130:411–452, 1983. doi: 10.1017/S0022112083001159.

[31] Robert H. Kraichnan. Inertial ranges in two-dimensional turbulence. *Physics of Fluids*, 10(7):1417–1423, 1967. doi: 10.1063/1.1762301.

[32] Guido Boffetta and Robert E. Ecke. Two-dimensional turbulence. *Annual Review of Fluid Mechanics*, 44: 427–451, 2012. doi: 10.1146/annurev-fluid-120710-101240.

[33] A. N. Kolmogorov. The local structure of turbulence in incompressible viscous fluid for very large Reynolds numbers. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, 434(1890): 9–13, 1991. doi: 10.1098/rspa.1991.0075. English translation; originally published in Dokl. Akad. Nauk SSSR **30**(4), 1941.

[34] Gideon Dresdner, Dmitrii Kochkov, Peter Norgaard, Leonardo Zepeda-Núñez, Jamie A. Smith, Michael P. Brenner, and Stephan Hoyer. Learning to correct spectral methods for simulating turbulent flows, 2022.

[35] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21): e2101784118, 2021. doi: 10.1073/pnas.2101784118.

[36] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal

Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax, 2018.

[37] Mark H. Carpenter and Christopher A. Kennedy. Fourth-order 2N-storage Runge–Kutta schemes. NASA Technical Memorandum 109112, NASA Langley Research Center, 1994.

[38] Mikael Mortensen. Shenfun: High performance spectral Galerkin computing platform. *Journal of Open Source Software*, 3(31):1071, 2018. doi: 10.21105/joss.01071.

[39] Mikael Mortensen, Lisandro Dalcin, and David Elliot Keyes. mpi4py-fft: Parallel fast Fourier transforms with MPI for Python. *Journal of Open Source Software*, 4(36):1340, 2019. doi: 10.21105/joss.01340.